

Introduction to Python

Assoc. Prof. Arthur Molnar

Babeş-Bolyai University

2025

Overview

1 Introduction to Python

- Data in Python
- Simple Data Types
- Compound Data Types
- Variables, expressions and statements

Hardware and software

- **Hardware** -*computers* (desktop, mobile, etc) and related *devices*
- **Software** -*programs* or *systems* which run on hardware
- **Programming language** - notation that defines syntax and semantics of programs

Python 101

- **Python** - a high level programming language. It is a great language for beginner programmers!
- **Python interpreter** - a program which allows us to run/interpret new programs.
- **Python standard library**: built-in functions and types

Python 101

Python 3 is:

- A modern programming language
- Simple to write and understand
- A **strongly typed** language
- A **dynamically typed** language
- An **interpreted** language
- A **garbage collected** language
- A language that supports multiple paradigms: structured, object-oriented, functional and aspect oriented programming
- A language with great support and many available libraries

Python 101

Python is...

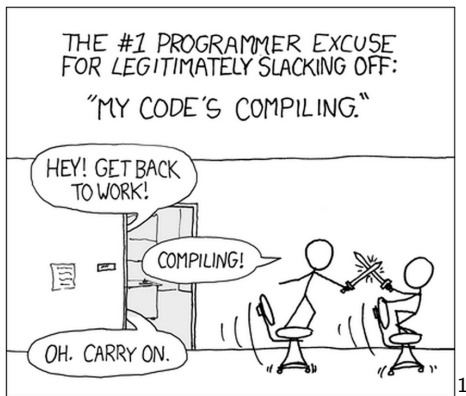
Simple to write and understand

```
1 myList = []  
2 while True:  
3     x = int(input("Enter item (-1 to finish):"))  
4     if x == -1:  
5         break  
6     myList.append(x)  
7 return myList
```

Python 101

Python is...

An **interpreted** language

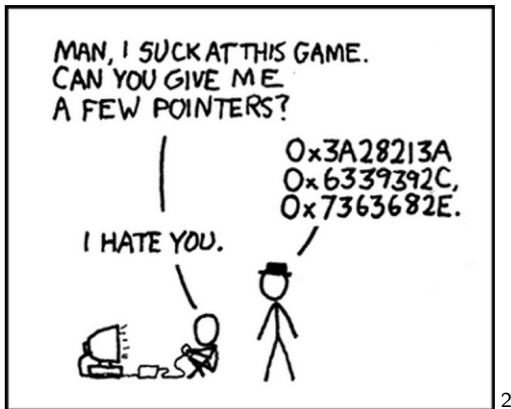


¹<https://xkcd.com/303/>

Python 101

Python is...

A **garbage collected** language



²<https://xkcd.com/138/>

Python 101

Python mantra³:

- Beautiful is better than ugly
- Explicit is better than implicit
- Simple is better than complex
- Flat is better than nested
- Sparse is better than dense
- Readability counts

³<https://www.python.org/dev/peps/pep-0020/>

What do you need?

- ① Python 3
- ② GitHub Desktop (**OR** use git in command line, **OR** git integration with an IDE)
- ③ An IDE (PyCharm **OR** Eclipse + PyDev **OR** VS Code)

What do you need?

Recommended configuration:

- Python 3 for your OS (from <https://www.python.org/>)
- PyCharm Professional (from <https://www.jetbrains.com/pycharm/>, create an educational profile using your UBB account at <https://www.jetbrains.com/shop/eform/students>)
- **[optional]** GitHub Desktop to clone lab assignment repositories (from <https://desktop.github.com/>)

Basic elements of a Python program

- **Lexical elements** - a Python program is divided into a number of **lines**.
- **Comments** - start with a hash (#) character and ends at the end of the line.
- **Identifiers** (or **names**) - are sequences of characters which start with a letter (a..z, A..Z) or an underscore (_) followed by zero or more letters, underscores, and digits (0..9).
- **Literals** - are notations for constant values of some built-in types.

Demo

Basic elements of a Python program

[ex01_basic_syntax.py](#)^a

^aProTip: This is a clickable link

Python data model

All data in Python programs is represented by objects, an **object** being Python's abstraction for data.

An **object** has:

- an **identity** - we may think of it as the object's address in memory (check out the `id()` built-in function).
- a **type** - which determines the operations that the object supports and also defines the possible values (check out the `type()` built-in function).
- a **value**.

Types

- **Types** classify values. A type denotes a **domain** (a set of possible values) and **operations** on those values.
- **Numbers, strings, tuples** - are immutable, so once created, their values cannot be changed.

Types in Python 3

- Each variable has a well known type at every moment during the program's execution (Python is **strongly** typed).
- Variable type can change together with its value as part of variable assignment (Python is **dynamically** typed).

Identity, value and type

Recall what is a *name* and an *object* (*identity*, *type*, *value*).

- mutable objects: lists, dictionaries, sets
- immutable: numbers, strings, tuples

We determine the identity and the type of an object using the built-in functions:

- `id(object)`
- `type(object)`
- `isinstance(object, type)` built-in function)

Standard types in Python (1/3)

int⁴:

- Represents the mathematical set of integers (positive and negative, unlimited precision)

bool:

- Represents the the truth values True and False.

float:

- Represents the mathematical set of double precision floating point numbers.

⁴<https://docs.python.org/3/library/stdtypes.html>

Standard types in Python (2/3)

Sequence types⁵

- Finite ordered sets indexed by non-negative numbers
- Let `a` be a sequence.
 - `len(a)` returns the number of items
 - `a[0]`, `a[1]`, ..., `a[len(a)-1]` represent the set of items
- These are the **string**, **list**, **tuple** and **range** objects⁶

string

- A string is an immutable sequence
- The items of a string are Unicode characters

⁵<https://docs.python.org/3/library/stdtypes.html#sequence-types-list-tuple-range>

⁶There are additional ones for processing binary data that remain outside the scope of our course

Standard types in Python (3/3)

list⁷

- Mutable sequence of elements
- Typically used to store collections of homogeneous items
- Every item has a predecessor and successor

tuple⁸

- Immutable sequence
- Typically used to store collections of homogeneous items

dict⁹

- Mapping between unique keys and values

⁷<https://docs.python.org/3/library/stdtypes.html#list>

⁸<https://docs.python.org/3/library/stdtypes.html#tuple>

⁹<https://docs.python.org/3/library/stdtypes.html#dict>

Demo

Basic compound types

[ex02_basic_compound_types.py](#)^a

^aProTip: This is a clickable link

List

Lists represent finite ordered sets indexed by non-negative numbers.

Operations:

- Creation
- Accessing values (index, len), changing values (**lists are mutable**)
- Removing items (pop), inserting items (insert)
- Slicing
- Nesting
- Generate list using range(), list in a for loop
- Lists as stacks (append, pop)

Tuple

Tuples are immutable sequences. A **tuple** consists of a number of values separated by commas.

Operations:

- Packing values (creation)
- Nesting
- Empty tuple
- Tuple with one item
- Sequence unpacking

Dictionary

A **dictionary** is an unordered set of (key, value) pairs with unique keys. The keys must be immutable.

Operations:

- Creation
- Getting the value associated to a given key
- Adding/updating a (key, value) pair
- Removing an existing (key, value) pair
- Checking whether a key exists

Python Data Structures Cheat Sheet

Python Data Structures Cheat Sheet

https:

[//www.stationx.net/python-data-structures-cheat-sheet/](https://www.stationx.net/python-data-structures-cheat-sheet/)

Variables and expressions

NB!

Variables are reserved memory locations to store values

- A **variable** has:
 - Name
 - Type
 - Domain
 - Operations

A variable is introduced in a program using a name binding operation - assignment.

Variables and expressions

- **Expression** - a combination of explicit *values*, *constants*, variables, *operators*, and *functions* that are interpreted according to the particular *rules of precedence*, which computes and then *produces/returns* another value.
- **Examples:**
 - numeric expression: $1 + 2$
 - boolean expression $1 < 2$
 - string expression: `'1' + '2'`

Statements

NB!

Statements are the basic operations of a program. A program is a sequence of statements

Statements

● Assignment

- Assignments are used to (re)bind names to values
- Bind name:
 - $x = 1$ *#is a variable (of type int)*
- Rebind name:
 - $x = x + 2$ *#a new value is assigned to x*
- Rebind name of mutable sequences:
 - $y = [1, 2]$ *#mutable sequence*
 - $y[0] = -1$ *#the first item is bound to -1*

● Block

- A block is a section of a program that is executed as a unit
- A sequence of statements is a block
- Blocks of code are denoted by line indentation

Demo

Controlling program flow

[ex03_program_flow.py](#)^a

^aProTip: This is a clickable link